

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class HebbianConv2d(nn.Module):
    def __init__(self, in_channels, out_channels,
kernel_size, stride=1, padding=0):
        super().__init__()
        self.conv = nn.Conv2d(in_channels,
out_channels, kernel_size,
                        stride=stride,
padding=padding, bias=False)

    def forward(self, x):
        return self.conv(x)

    def local_hebbian_update(self, x, y, lr=0.01):
        # x: [B, C_in, H, W], y: [B, C_out, H', W']
        b, cin, h, w = x.shape
        cout = y.shape[1]

        # Extract patches для каждого фильтра
```

```

for o in range(cout):
    for i in range(cin):
        # Patches: [B, kH*kW, H', W']
        patches = F.unfold(x[:, i:i+1],
self.conv.kernel_size,
                        self.conv.stride,
self.conv.padding)

        # Local Hebbian: patches × output
        delta_k = lr * (patches * y[:,
o:o+1]).mean(dim=0)
        delta_k =
delta_k.view(self.conv.kernel_size[0],
                self.conv.kernel_size[1],
-1)

        # Oja normalization
        kernel_norm = torch.norm(y[:, o],
dim=[2,3], keepdim=True)
        self.conv.weight.data[o, i] += delta_k
* (1.0 - kernel_norm)

```

```
def oja_normalize(self):  
    # Нормализация по выходу (Oja's rule)  
    with torch.no_grad():  
        self.conv.weight.data *= 0.995 # decay
```

Пример использования

```
model = nn.Sequential(  
    HebbianConv2d(3, 32, 3, padding=1),  
    nn.ReLU(),  
    HebbianConv2d(32, 64, 3, padding=1),  
    nn.ReLU()  
)
```

```
x = torch.randn(4, 3, 32, 32)  
y_pred = model(x)
```

Онлайн Hebbian обучение

```
for layer in model:  
    if isinstance(layer, HebbianConv2d):  
        layer.local_hebbian_update(x, y_pred,  
lr=0.01)  
        layer.oja_normalize()
```